
Polyp Segmentation using ResUNet++ on Kvasir-SEG

Submitted By:

Writick Parui — 8025320111
Sougata Mukherjee — 8025320095
Shreya Srivastava — 8025320091

Submitted To:

Dr. Abhishek Kesarwani
(Assistant Professor)



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Computer Science & Engineering Department
Thapar Institute of Engineering & Technology, Patiala
May 2026

Course / Class	PCS 220: Multimedia Processing Lab
Semester	Second Semester, 2025–26
Submission Date	01 May 2026
Supervisor	Dr. Abhishek Kesarwani (Assistant Professor)

Abstract

Colorectal cancer is one of the leading causes of cancer-related mortality worldwide, and most colorectal cancers develop from precursor polyps. Colonoscopy is the standard screening procedure, but manual detection and boundary localisation can be affected by fatigue, camera motion, specular highlights, mucosal folds, variable illumination and the large visual diversity of polyp shapes. Automated semantic segmentation can assist by producing a pixel-level mask of suspected polyp regions, making the output more interpretable than image-level classification.

This project implements an updated deep-learning pipeline for polyp segmentation on the Kvasir-SEG dataset. The final model is ResUNet++ rather than the older Multi-Scale U-Net prototype described in the earlier report. The architecture combines residual encoder-decoder blocks, squeeze-and-excitation channel recalibration, an Atrous Spatial Pyramid Pooling (ASPP) bridge, attention-gated skip connections and deep-supervision heads named output, ds3 and ds2. These components help the network preserve boundary detail, capture context across multiple polyp sizes and suppress irrelevant background features from skip connections.

The data pipeline resizes images and masks to 352 x 352, normalises images to [0, 1], and uses a 12-transform Albumentations augmentation pipeline including flips, rotation, ShiftScaleRotate, ElasticTransform, GridDistortion, OpticalDistortion, CLAHE, colour/intensity variation, Gaussian noise and CoarseDropout. Training is performed for 60 epochs with mixed precision and a warmup-cosine learning-rate schedule, without EarlyStopping. The best validation checkpoint achieves $\text{val_output_dice_coef} = 0.7695$, $\text{val_output_iou_metric} = 0.6291$ and $\text{val_loss} = 0.7767$ at epoch 58. Final evaluation on the rebuilt 100-sample test set gives single-pass Dice = 0.7607 and IoU = 0.6612, while 8-fold test-time augmentation improves Dice to 0.7777 and IoU to 0.6825.

Framework	TensorFlow / Keras
Dataset	Kvasir-SEG (1,000 images and masks)
Model	ResUNet++ with ASPP + Attention Gates
Input Size	352 x 352
Best Validation Dice / IoU	0.7695 / 0.6291 at epoch 58
Final TTA Test Dice / IoU	0.7777 / 0.6825

1. Introduction

1.1 Background and Motivation

Colorectal cancer is highly preventable when precursor polyps are detected early and removed. In practice, however, colonoscopy images contain noise, glare, folds, blurry frames and strong appearance variation. A segmentation model must distinguish polyp tissue from visually similar surrounding mucosa and produce a mask that follows the lesion boundary.

Semantic segmentation is especially useful in this setting because the output is a binary region rather than a simple class label. This can support localisation, approximate measurement, visual explanation and downstream clinical review.

1.2 Problem Statement

The task is binary semantic segmentation of Kvasir-SEG colonoscopy images. The latest project uses ResUNet++ with residual blocks, SE recalibration, ASPP context extraction and attention-gated decoder skips. The model is trained and evaluated using Dice, IoU, single-pass inference and 8-fold test-time augmentation.

1.3 Objectives

- Implement a ResUNet++ model for polyp segmentation using 352 x 352 inputs.
- Use Albumentations augmentation to handle geometric, colour and occlusion variability.
- Train for all 60 epochs with warmup-cosine scheduling and checkpointing.
- Evaluate best_model.keras on a rebuilt 100-sample test set.
- Report validation metrics, single-pass test metrics, 8-fold TTA metrics and qualitative prediction grids.

1.4 Sample Colonoscopy Images

The following raw colonoscopy samples are attached to show the visual diversity handled by the project. The examples include small sessile polyps, larger protruding lesions, different mucosal textures, specular highlights and black endoscopic borders. This variation motivates the use of 352 x 352 preprocessing, Albumentations augmentation and a ResUNet++ architecture with ASPP and attention gates.



2. Literature Review

2.1 Classical Approaches

Early computer-aided polyp detection methods used hand-crafted features such as colour histograms, edge descriptors, texture measures, local binary patterns, region growing and threshold-based segmentation. These approaches were lightweight and interpretable, but they were sensitive to illumination changes, specular highlights, fluid, folds and camera viewpoint. Because polyp appearance is highly variable, fixed feature rules often failed to generalise across patients and acquisition conditions.

2.2 Deep Learning for Medical Segmentation

Fully convolutional networks enabled end-to-end dense prediction by replacing classification heads with convolutional upsampling. U-Net then became a dominant biomedical segmentation architecture because it combines an encoder for semantic abstraction with a decoder for spatial recovery and skip connections for fine detail. U-Net is particularly suitable for medical datasets where annotation is limited, because data augmentation and skip-connected decoding help preserve structure.

However, vanilla U-Net can still struggle with small lesions, ambiguous boundaries and scale variation. Modern medical segmentation systems often add residual learning, attention, multi-scale context modules and deep supervision to improve optimisation and localisation quality.

2.3 U-Net Variants for Polyp Segmentation

- **ResUNet:** introduces residual blocks to improve gradient flow and stabilise deeper encoder-decoder training.
- **Attention U-Net:** uses attention gates to suppress irrelevant skip-connection features before concatenation.
- **Double U-Net:** stacks two U-Net-like modules and uses ASPP to strengthen multi-scale representation.
- **PraNet:** applies reverse attention and partial decoder mechanisms for polyp segmentation refinement.
- **TransFuse and Polyp-PVT:** combine convolutional local features with transformer-style global context.
- **ResUNet++:** combines residual blocks, squeeze-and-excitation, ASPP and attention mechanisms, making it well matched to this project.

2.4 Multi-Scale Feature Learning

Polyps vary from tiny spots to large protruding lesions, so multi-scale context is crucial. The latest project uses ASPP at the bridge, where parallel atrous convolutions with different dilation rates capture contextual information without excessively reducing spatial resolution. This is the correct latest-model explanation and replaces the previous raw downsampled input scale-stream description.

3. Dataset - Kvasir-SEG

3.1 Overview

Kvasir-SEG is a public benchmark dataset for gastrointestinal polyp segmentation. It contains colonoscopy images with pixel-level binary masks, making it suitable for supervised semantic segmentation. The images vary in resolution, lesion size, texture, colour and field-of-view quality.

Property	Details
Total Images	1,000 colonoscopy images
Total Masks	1,000 pixel-level binary segmentation masks
Image Format	JPEG-style files loaded from images folder
Image Resolution	Variable original resolution; resized to 352 x 352
Annotation	Binary foreground/background mask
Source	Simula Research Laboratory, Norway

3.2 Data Split

The notebook uses `train_test_split` with `SEED = 42`. Ten percent of samples are reserved for testing, then the remaining samples are split so validation represents 15 percent of the full dataset.

Split	Proportion	Approx. Samples	Use
Training Set	75%	~750	Model optimisation and augmentation
Validation Set	15%	~150	Checkpoint selection
Test Set	10%	~100	Single-pass and TTA evaluation

3.3 Preprocessing & Augmentation

- Images are read with OpenCV, resized to 352 x 352, converted from BGR to RGB and normalised to [0, 1].
- Masks are read as grayscale, resized using nearest-neighbour interpolation and expanded to one channel.
- Training augmentation includes flips, 90-degree rotation, ShiftScaleRotate, ElasticTransform, GridDistortion, OpticalDistortion, CLAHE, HueSaturationValue, RandomBrightnessContrast, GaussNoise and CoarseDropout.
- Validation and test splits use deterministic preprocessing only, so metric reporting is consistent.

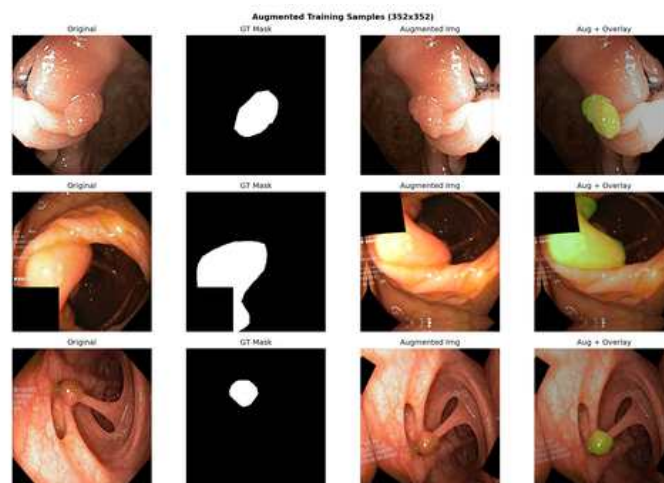


Figure 2: Latest augmented training examples.

4. Methodology

4.1 Model Architecture - ResUNet++

The latest model is a ResUNet++ style encoder-decoder. It starts with a convolution stem, uses residual blocks with squeeze-and-excitation in the encoder, applies ASPP in the bridge, and decodes through transposed convolutions with attention-gated skip connections. The model has three outputs: output, ds3 and ds2.

Stage	Type	Filters	Output Size
Input	RGB image	-	352 x 352 x 3
Stem	Conv + BN + ReLU	32	352 x 352
Encoder 1	Residual + SE + Pool	32	176 x 176
Encoder 2	Residual + SE + Pool	64	88 x 88
Encoder 3	Residual + SE + Pool	128	44 x 44
Encoder 4	Residual + SE + Pool	256	22 x 22
Bridge	ASPP	512	22 x 22
Decoder 4	Upsample + Attention + Residual	256	44 x 44
Decoder 3	Upsample + Attention + Residual	128	88 x 88
Decoder 2	Upsample + Attention + Residual	64	176 x 176
Decoder 1	Upsample + Attention + Residual	32	352 x 352
Outputs	1 x 1 Conv + Sigmoid	1	352 x 352 x 1

4.2 Loss Function

The notebook defines Focal-Tversky loss and Boundary loss. Focal-Tversky penalises false negatives strongly, which is important because missing a polyp region is clinically more serious than a small false positive. Boundary loss adds an edge-aware penalty near ground-truth mask borders. The custom combined loss is $0.8 \times \text{Focal-Tversky} + 0.2 \times \text{Boundary}$. The XLA-safe training cell recompiles the multi-output model and includes a Dice+BCE-style fallback loss_fn when needed.

4.3 Evaluation Metrics

Metric	Meaning	Ideal
Dice Coefficient	Overlap score: $2 \times P \cap G / (P + G)$	1.0
IoU	Jaccard overlap: $ P \cap G / P \cup G $	1.0
Loss	Segmentation optimisation objective	0.0

4.4 Training Configuration

Hyperparameter	Latest Notebook Value
Input Size	352 x 352 x 3
Batch Size	8
Epochs	60 full epochs; no EarlyStopping
Learning Rate	Warmup + cosine schedule
Warmup	5 epochs
LR Range	2e-4 peak, 1e-6 / 1e-7 final floor depending cell
Base Filters	32
Optimizer	Adam
Mixed Precision	mixed_float16 policy
Checkpoint	best_model.keras monitored by val_output_dice_coef
Logging	training_log.csv

4.5 Learning Rate Schedule

The learning rate rises linearly during warmup and then follows cosine decay. This replaces the old ReduceLROnPlateau/EarlyStopping language. The model trains for the full 60 epochs and saves the best checkpoint separately.

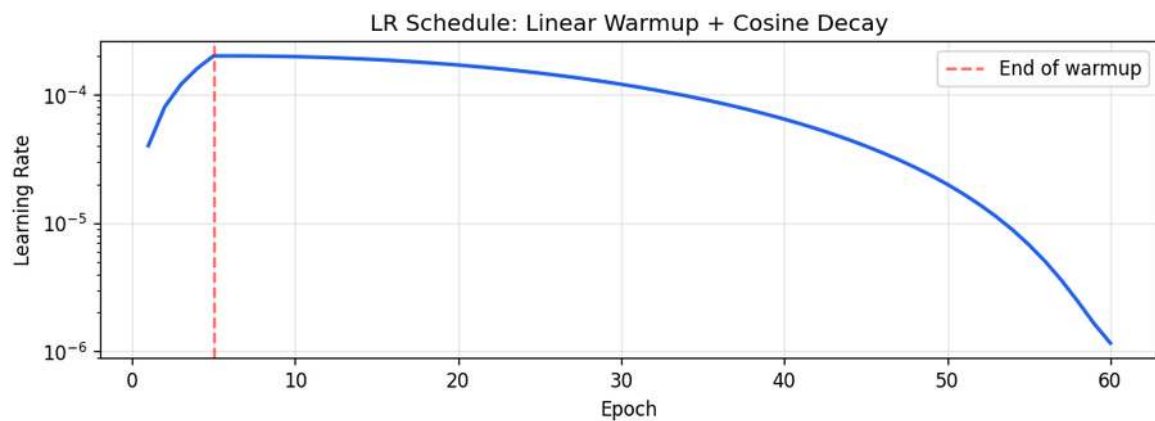


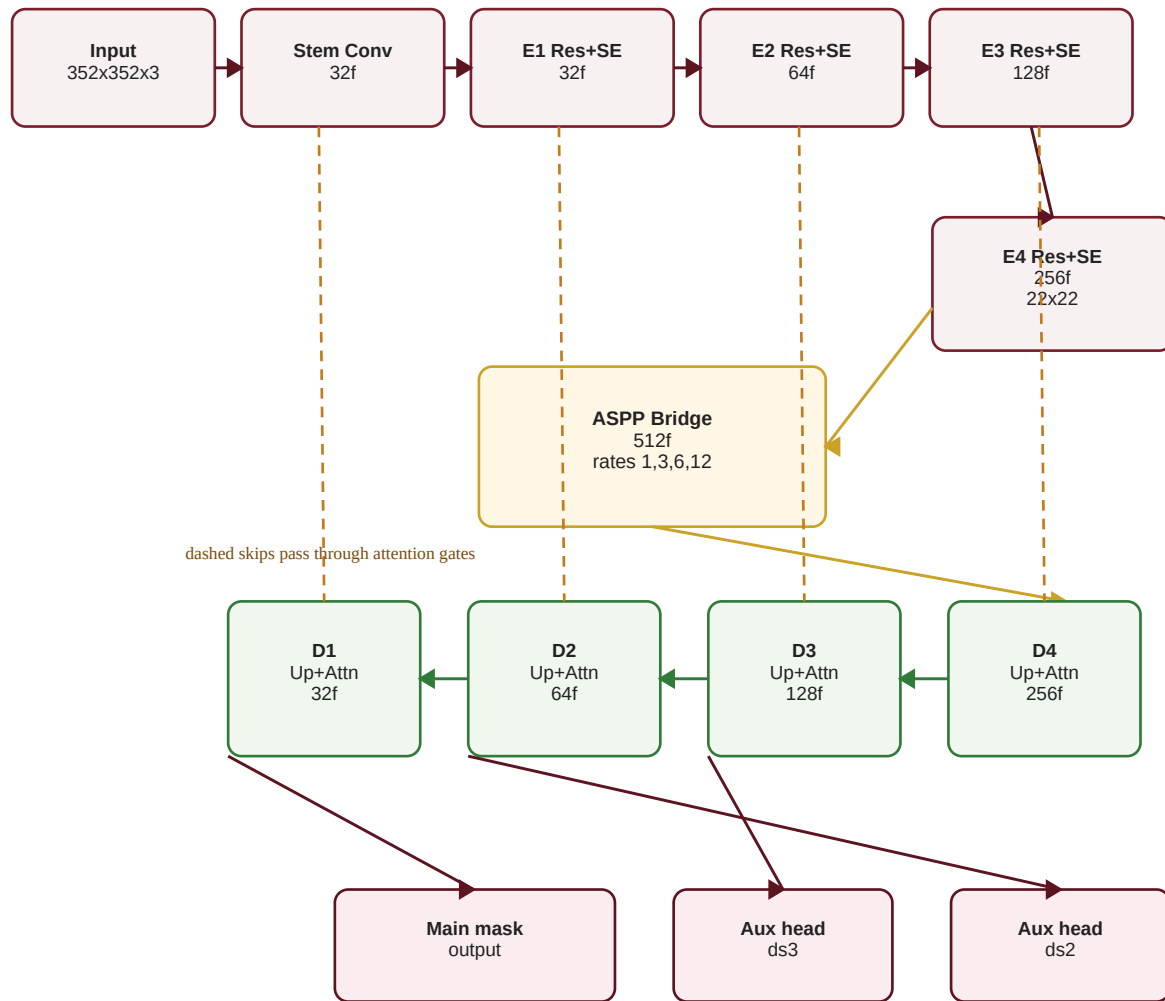
Figure 3: Linear warmup and cosine decay learning-rate schedule.

4.6 Deep Supervision

The output names are ['output', 'ds3', 'ds2']. The main head is used for final reporting, while ds3 and ds2 provide auxiliary supervision to intermediate decoder stages, improving gradient flow during training.

4.7 Architecture Diagram - ResUNet++

ResUNet++ with ASPP Bridge, Attention Gates and Deep Supervision



All heads produce sigmoid 352 x 352 x 1 masks; final metrics use the main output.

Figure 4: ResUNet++ architecture diagram with residual+SE encoder, ASPP bridge, attention-gated decoder and deep supervision heads.

5. Implementation Details

5.1 Technology Stack

Component	Tool / Library	Version / Note
Language	Python	3.10+
Deep Learning	TensorFlow / Keras	2.x
Augmentation	Albumentations	1.3.1
Image Processing	OpenCV	cv2
Data Pipeline	tf.data API	AUTOTUNE
Splitting	scikit-learn	train_test_split
Visualisation	Matplotlib	PNG output artifacts
Compute Platform	Google Colab	GPU runtime

5.2 Data Pipeline

- The dataset zip is uploaded and extracted in Colab; image and mask folders are auto-detected.
- File paths are split into train, validation and test sets using a fixed random seed.
- `tf.numpy_function` wraps OpenCV and Albumentations preprocessing inside `tf.data`.
- Training data is shuffled, mapped, batched and prefetched.
- For multi-output training, each ground-truth mask is supplied to output, `ds3` and `ds2`.

5.3 Notebook Structure (14+ Cells)

Cell	Description
1	Upload and extract kvasir-seg.zip; auto-detect images and masks folders
2	Install/import libraries, set random seeds and enable mixed precision
3	Configuration: IMG_SIZE=352, BATCH_SIZE=8, EPOCHS=60, LR and output paths
4	Load image/mask paths and create train/validation/test split
5	Define 12-transform Albumentations training augmentation pipeline
6	Build tf.data datasets with OpenCV preprocessing, batching and prefetching
7	Visualise sample/augmented training images and save artifacts
8	Define ResUNet++ with residual blocks, SE, ASPP and attention gates
9	Define Dice/IoU metrics, Focal-Tversky loss, Boundary loss and compile
10	Create and plot warmup + cosine learning-rate schedule
11	Train full 60 epochs with ModelCheckpoint and CSVLogger
12	Plot training curves from training_log.csv
13	Rebuild model, load best_model.keras, evaluate single-pass and 8-fold TTA
14	Export Hugging Face deployment package and generated outputs

6. Results and Evaluation

6.1 Training History

Training completed all 60 epochs. The best validation checkpoint is identified from training_log.csv. Validation Dice, validation IoU and validation loss all point to epoch 58 as the strongest epoch for the final reported checkpoint.

Metric	Best Value	Epoch
val_output_dice_coef	0.7695	58
val_output_iou_metric	0.6291	58
val_loss	0.7767	58
val_output_loss	0.4291	58



Figure 5: Latest training history showing total loss, Dice, IoU and main-output loss.

6.2 Quantitative Comparison

Model	Dice	IoU	Reference
Standard U-Net	0.818	0.746	Jha et al., 2020
ResU-Net	0.813	0.793	Zhang et al., 2018
Double U-Net	0.813	0.733	Jha et al., 2020
PraNet	0.898	0.840	Fan et al., 2020
This Project (ResUNet++)	0.7695 (Val)	0.6291 (Val)	This Work
This Project + 8-fold TTA	0.7777 (Test)	0.6825 (Test)	This Work

6.3 Test Results

Evaluation	Dice	IoU
Single-pass mean +/- std	0.7607 +/- 0.2352	0.6612 +/- 0.2541
8-fold TTA mean +/- std	0.7777 +/- 0.2302	0.6825 +/- 0.2500
TTA improvement	+1.70% Dice	-

6.4 Qualitative Results

The prediction grid shows original image, ground truth, predicted mask and overlay. It includes both strong predictions and difficult cases, giving a realistic view of model behaviour. The strongest examples show clean overlap, while harder cases reveal fragmentation or over-segmentation near folds and highlights.

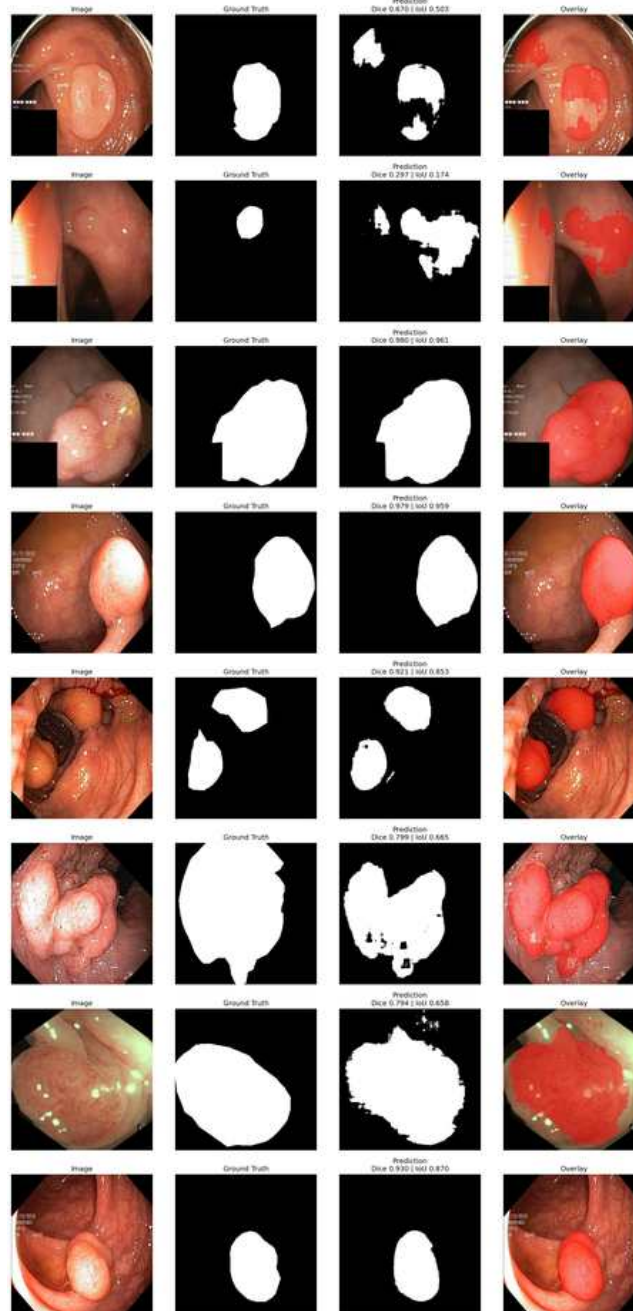


Figure 6: Latest prediction grid from final model outputs.

7. Discussion

7.1 Strengths

- Residual blocks improve gradient flow and enable deeper feature extraction.
- Squeeze-and-excitation recalibrates channels based on global context.
- ASPP captures multiple receptive-field scales at the bottleneck.
- Attention gates reduce irrelevant skip-connection background features.
- Albumentations augmentation improves robustness to endoscopy variation.
- Warmup-cosine training runs all 60 epochs without early stopping.
- 8-fold TTA improves Dice by 1.70 percentage points on the test set.

7.2 Limitations

- The dataset contains only 1,000 labelled images, so generalisation is limited by dataset size.
- The standard deviation of test Dice and IoU is high, indicating difficult outliers.
- A fixed 0.5 threshold may not be optimal for all samples.
- Some masks are fragmented or leak into nearby tissue in challenging images.
- No external validation is reported on CVC-ClinicDB, ETIS-LaribPolypDB or ColonDB.

7.3 Future Work

- Calibrate threshold using validation data.
- Add connected-component filtering and morphology smoothing.
- Evaluate external datasets for domain generalisation.
- Try pretrained encoders such as ResNet-50, EfficientNet-B4 or ConvNeXt.
- Optimise inference for real-time colonoscopy video.

8. Conclusion

This project presents a complete ResUNet++ pipeline for automated polyp segmentation on Kvasir-SEG. The report has been updated to reflect the actual latest notebook implementation: 352 x 352 inputs, residual+SE blocks, ASPP bridge, attention gates, deep supervision, Albumentations augmentation, mixed precision, warmup-cosine training and 8-fold TTA.

The model achieves best validation Dice 0.7695, validation IoU 0.6291 and validation loss 0.7767 at epoch 58. On the 100-sample test set, 8-fold TTA improves the final Dice to 0.7777 and IoU to 0.6825. These results show that the upgraded ResUNet++ system substantially improves the older prototype terminology and provides a stronger basis for deployment.

8.1 Final Metric Summary

Metric	Value
Best validation Dice	0.7695 at epoch 58
Best validation IoU	0.6291 at epoch 58
Best validation loss	0.7767 at epoch 58
Single-pass test Dice / IoU	0.7607 / 0.6612
8-fold TTA test Dice / IoU	0.7777 / 0.6825

References

- [1] Jha, D. et al. (2020). Kvasir-SEG: A Segmented Polyp Dataset. MMM 2020.
- [2] Ronneberger, O. et al. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI 2015.
- [3] Fan, D. P. et al. (2020). PraNet: Parallel Reverse Attention Network for Polyp Segmentation. MICCAI 2020.
- [4] Jha, D. et al. (2020). DoubleU-Net: A Deep Convolutional Neural Network for Medical Image Segmentation. CBMS 2020.
- [5] Long, J. et al. (2015). Fully Convolutional Networks for Semantic Segmentation. CVPR 2015.
- [6] Milletari, F. et al. (2016). V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation. 3DV 2016.
- [7] Zhang, Z. et al. (2018). Road Extraction by Deep Residual U-Net. IEEE Geoscience and Remote Sensing Letters, 15(5).
- [8] Abadi, M. et al. (2016). TensorFlow: A System for Large-Scale Machine Learning. OSDI 2016.
- [9] Alumentations documentation and latest submitted notebook implementation artifacts.

9. Deployment

9.1 Overview

The trained ResUNet++ model is deployed as a browser-accessible Gradio application hosted on Hugging Face Spaces. The live interface allows a user to upload a colonoscopy image and returns two outputs: a binary mask and a red overlay showing the predicted polyp region.

The deployment keeps the project usable without a local Python environment or GPU. Model training, architecture design and evaluation are based on the latest notebook; the interface provides an accessible demonstration for clinicians, researchers and evaluators.

Live Deployment URL	https://huggingface.co/spaces/Writick/polyp-segmentation
----------------------------	---

9.2 Deployment Interface

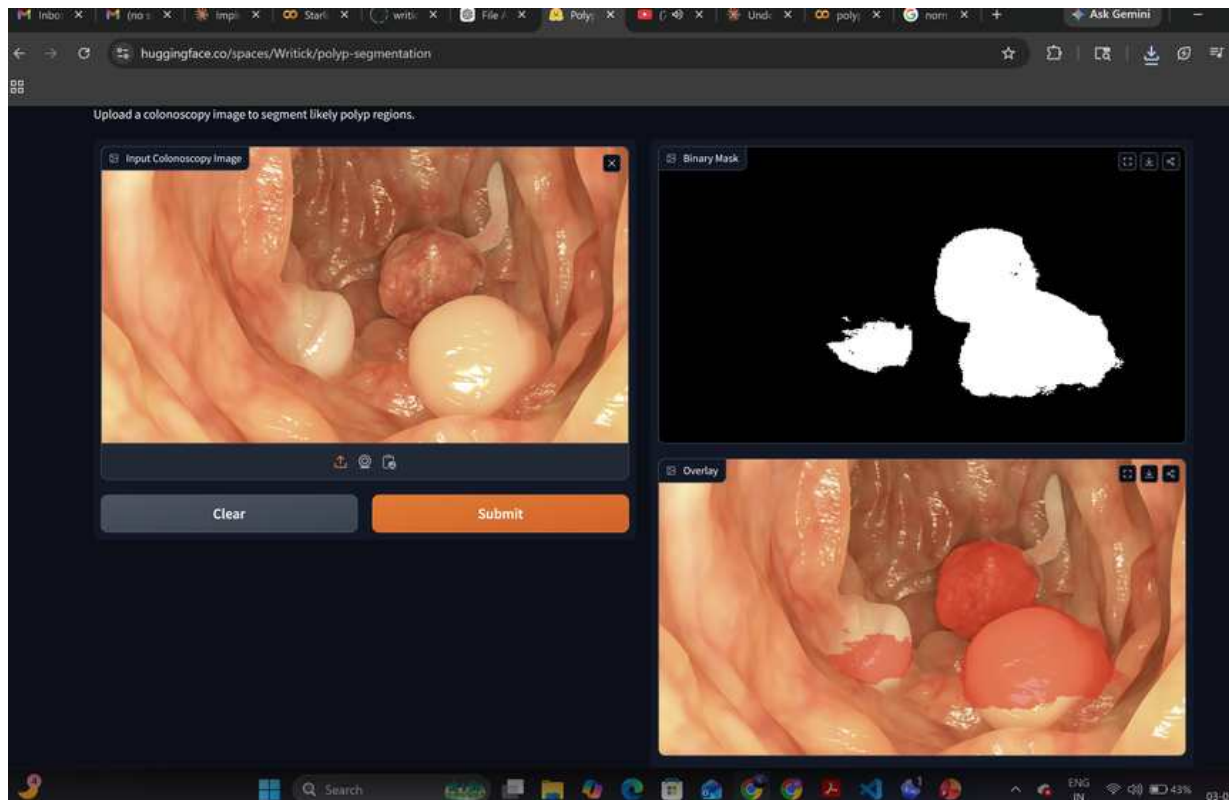


Figure 7: Desktop deployment view showing input image, binary mask and overlay output.

9. Deployment (continued)

9.2 Deployment Interface

The interface accepts a colonoscopy JPEG/PNG input. The backend preprocesses the image to the model resolution, performs segmentation, thresholds the probability mask and returns both a binary mask and an overlay. The overlay helps visually inspect whether predicted regions correspond to visible polyps.

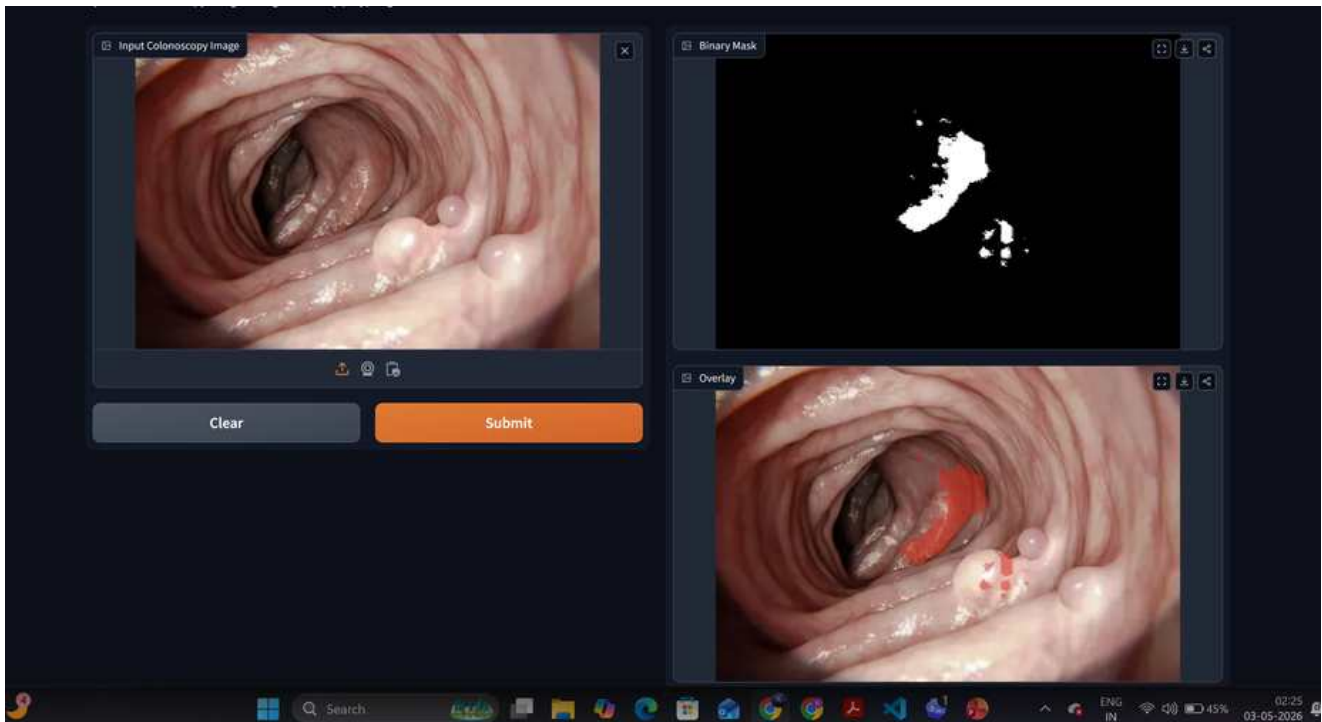


Figure 8: Additional deployment example with generated binary mask and red overlay.

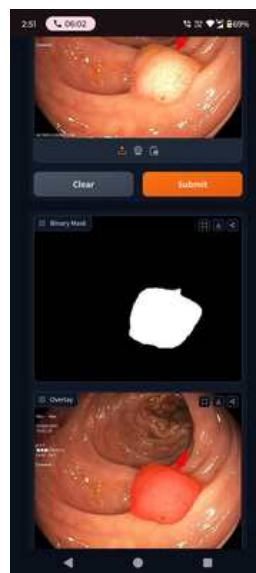


Figure 9.3: Mobile view of the Hugging Face Spaces deployment from the previous report.

9.3 Contribution Summary

Task	Contributor(s)
ResUNet++ architecture design	Writick Parui, Sougata Mukherjee
Model training & hyperparameter tuning	Writick Parui, Sougata Mukherjee
Gradio interface development & integration	Writick Parui
Hugging Face Spaces deployment & hosting	Writick Parui
Deployment testing & inference validation	Writick Parui, Sougata Mukherjee
Report write-up & documentation	Writick Parui, Sougata Mukherjee, Shreya Srivastava

9.4 Deployment Notes

The deployed Gradio application demonstrates practical use of the segmentation model. A user uploads a colonoscopy frame, clicks submit, and receives the predicted segmentation mask. The binary mask makes foreground/background separation clear, while the red overlay communicates the prediction directly on the input image.

Because medical images can contain multiple suspicious regions, the model may output multiple connected components. This is visible in deployment examples where separate predicted blobs appear in the binary mask and overlay. In a future production workflow, connected-component filtering, confidence scoring and threshold calibration could be added before presenting the result to a clinician.

The deployment is intended for academic demonstration and project evaluation. It should not be used as a standalone clinical diagnostic tool without broader validation, regulatory review, clinical testing and integration into a supervised workflow.

Deployment Feature	Description
Input	Colonoscopy JPEG/PNG image
Output 1	Binary mask: white polyp foreground on black background
Output 2	Overlay: red predicted region superimposed on input
Hosting	Hugging Face Spaces
Interface	Gradio
Model family	ResUNet++ segmentation model

Appendix: Project Artifacts and Reproducibility

The final report is based on the provided notebook, trained model, CSV log and generated visual outputs. The artifacts together make the project reproducible: the notebook defines the preprocessing, augmentation, architecture, loss functions, training schedule, checkpointing, evaluation and deployment export steps; the CSV log verifies the reported validation metrics; the PNG figures document samples, augmentation, learning-rate schedule, training curves and qualitative predictions.

Artifact	Purpose in Report
polyp_seg_max_accuracy_1 (2).ipynb	Main source of latest model architecture, data pipeline, training and evaluation code
best_model.keras	Best checkpoint loaded for final single-pass and 8-fold TTA evaluation
training_log.csv	Source for validation Dice, IoU, loss and epoch-wise training curves
sample_data.png	Dataset visualisation: image, mask and ground-truth overlay
augmented_samples.png	Albumentations pipeline visualisation
lr_schedule.png	Warmup + cosine learning-rate schedule
training_curves.png	Training/validation loss, Dice, IoU and main-output loss
prediction_grid.png	Qualitative segmentation examples
Deployment screenshots	Evidence of the Hugging Face Spaces Gradio interface

Reproducibility Notes

- Random seed is set to 42 for Python, NumPy and TensorFlow.
- The dataset split is deterministic using `train_test_split`.
- The model checkpoint is selected using `val_output_dice_coef`.
- Final evaluation rebuilds the model, loads weights from `best_model.keras` and evaluates a rebuilt 100-sample test set.
- 8-fold TTA averages flipped/rotated predictions before thresholding.